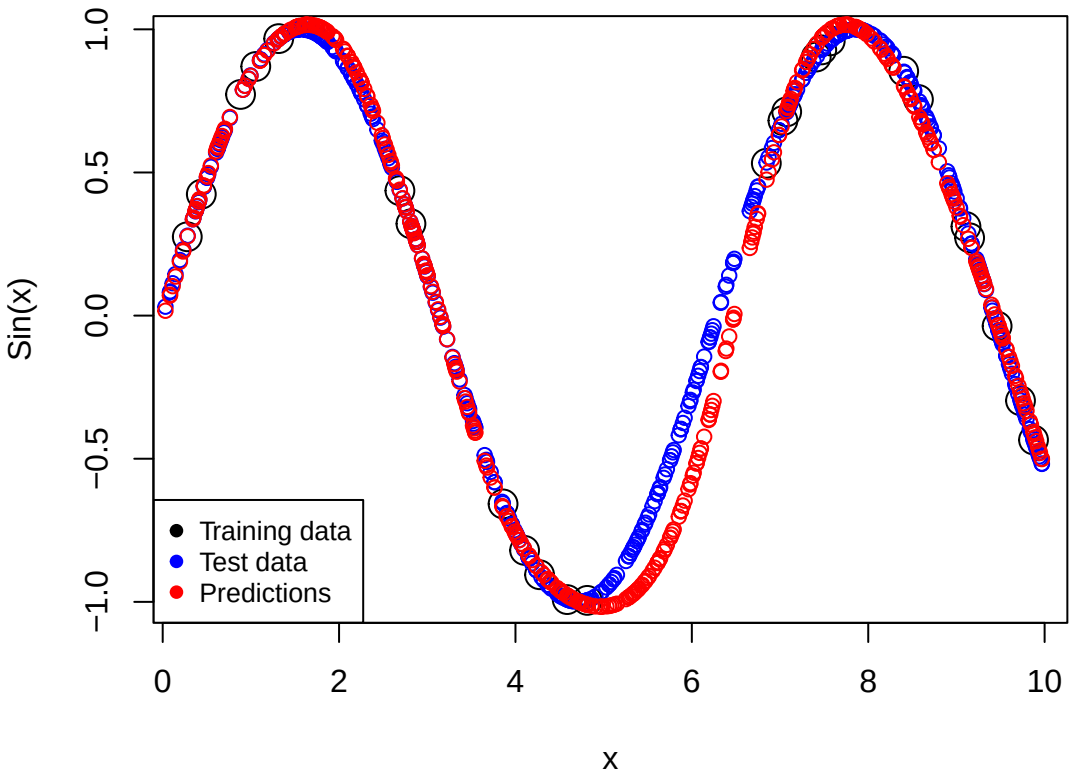
Lab3 assignment4

2025-12-11

Part 1 - Train NN on trigonometric sine function

We train a standard neural network with default settings to predict the output of a trigonometric sine function. (The default NN uses a sigmoid activation function). To do so we sample 500 points uniformly at random in the interval [0,10] and apply the sine function to each point. 25 of the points are used for training and the rest for test. The following plot shows the learned NN on the test data.



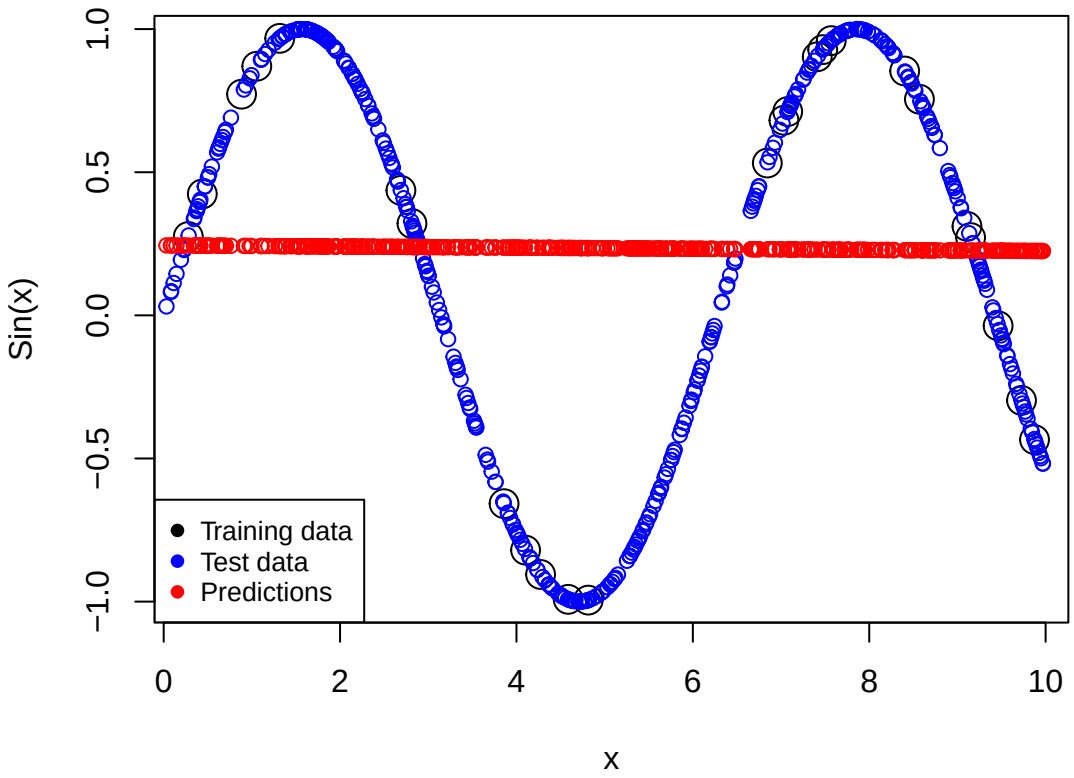
The predicted values of Sin(x) follow the training and test data quite well. There is a bigger chunk of worse predictions between x=5-7.

Part 2 - training with different activation functions

For part 2 of the assignment we train three NNs with different activation functions. The same training and test data is used as in part 1.

NN with linear activation function

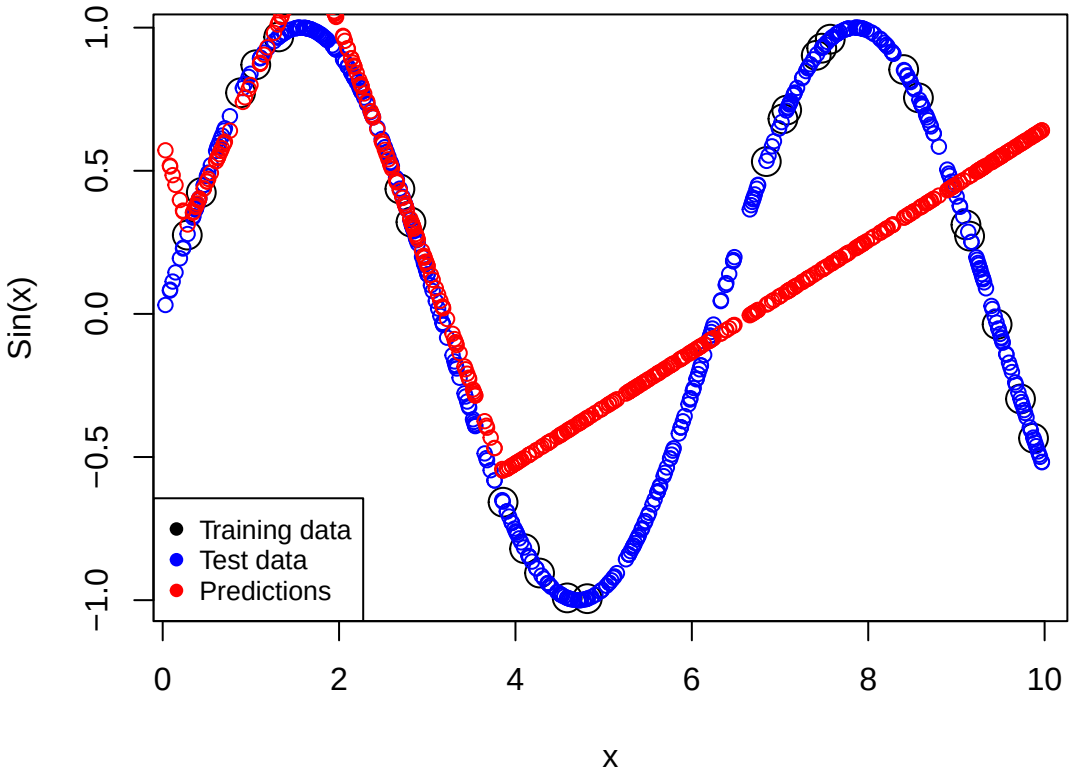
h(x) = x



The predicted values are a straight line following y ≈ 0.25. With a linear activation function the neural network behaves just like a linear regression which gives poor performance for our data.

NN with ReLU activation function

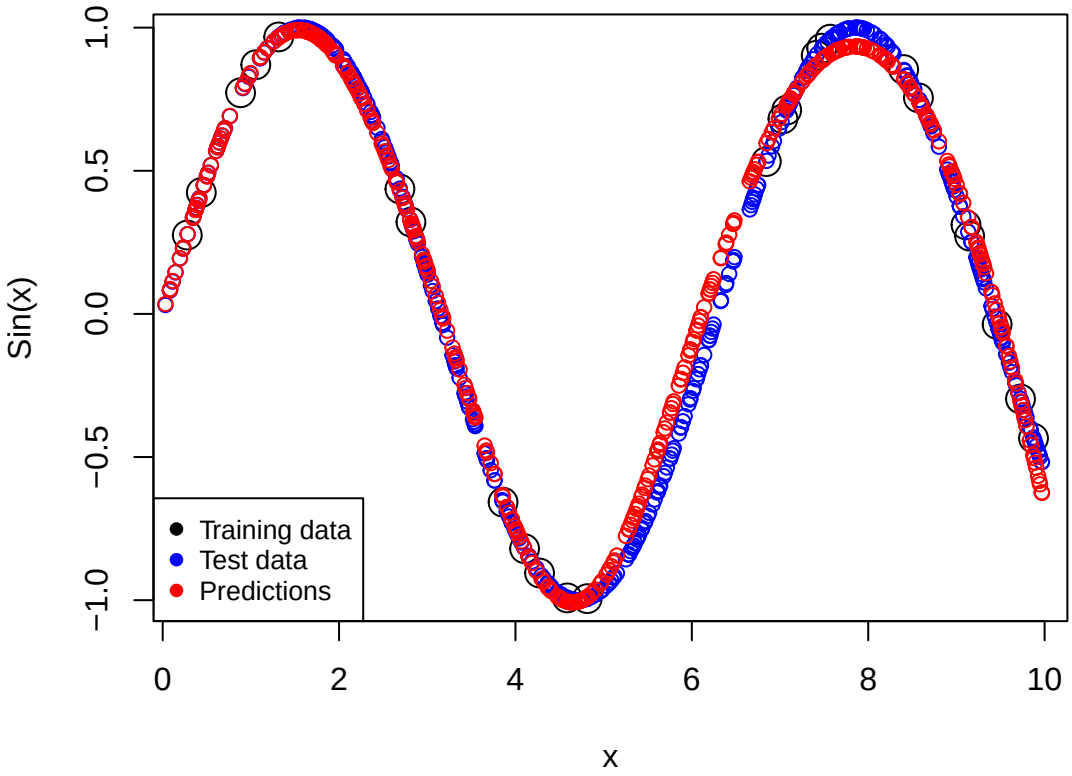
h(x) = max{0,x}



With the ReLU activation function we get relatively good performance between x=0.5 and x=4. The performance for other x is poor but still better than the NN with a linear activation function.

NN with softplus activation function

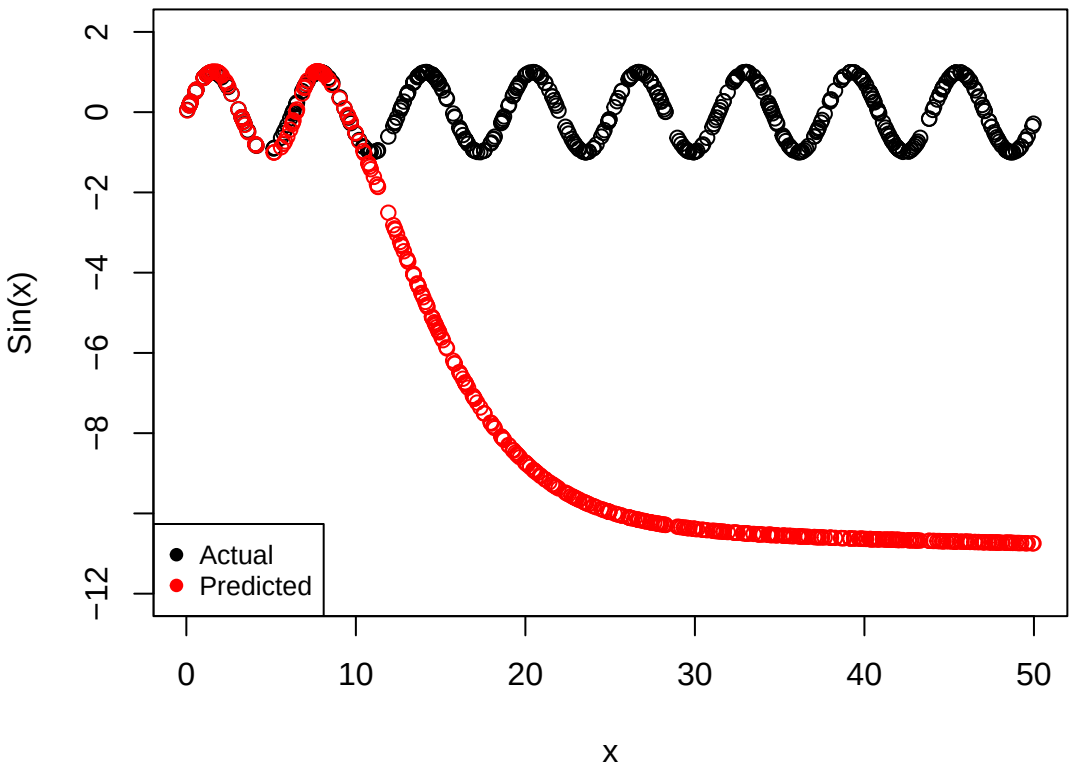
$h(x) = \ln(1 + \exp(x))$



With the softplus activation function we get the best performance out of all the above activation functions with the perdictions on the test data fitting the training data very well.

Part 3 and 4 - Prediction over bigger interval

We sample a new set of 500 points uniformly at random in the interval [0,50], apply the sine function to each point and train a new neural network. The NN uses a sigmoid activation function. The following plot shows the learned NN on the test data.



The model fits to the sine curve well for values of x inside the range of 0-10. As x grows beyond x=10 the predictions deviate towards a constant x ≈ -10.

This is because as the input grows the activation functions for the hidden units starts to saturate. The NN used in part 3 uses a sigmoid activation function. As the value of X for a sigmoid increases its output starts to approach "1" while when it decreases it approaches "0".

The following weights were learned during the training of the neural network:

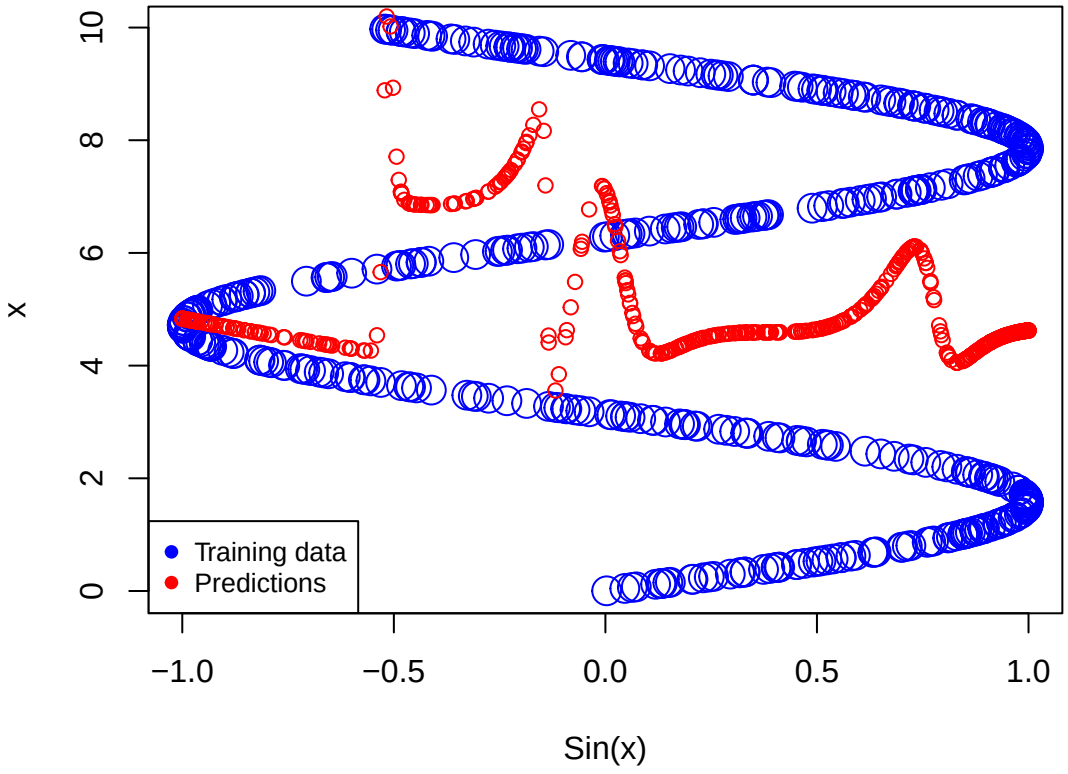
```
## [[1]]
## [[1]][[1]]
##            [,1]       [,2]       [,3]       [,4]      [,5]       [,6]      [,7]
## [1,] -11.870831 -0.9153178 -3.0316259  1.5313105  6.555437 -11.768525  1.397325
## [2,]   4.042494 -0.5368677  0.2603355 -0.5487656 -2.233839   1.787862 -1.259834
##            [,8]       [,9]       [,10]
## [1,]  0.2032402  0.2948804 -0.21326487
## [2,] -2.1974036 -0.5886005 -0.03177278
##
## [[1]][[2]]
##              [,1]
##  [1,]  -0.1088032
##  [2,]   0.7716196
##  [3,]  -1.0436754
```

```
##  [4,] -16.0529801
##  [5,]  -1.2718986
##  [6,]   3.2805756
##  [7,]   4.3076898
##  [8,]   0.1220703
##  [9,]  -1.2234185
## [10,]  -2.7618205
## [11,]   2.3696931
```

By looking at the learned weights we can see that some of the weights are relatively large which means that the input to the activation functions also will be large. Due to the nature of the sigmoid activation function, large inputs will quickly go towards either 0 or 1. When this happens the curve flattens.

Part 5 - Predict x from Sin(x)

In the last part of the lab we attempt to train a neural network to predict "x" from the output of Sin(x). Once again we sample 500 points uniformly at random in the interval [0,10], apply the sine function to each point and train the neural network. This neural network also uses the sigmoid activation function. This gives the following plot:



From the plot we can tell that the neural network is having a very hard time with its predictions. This is

because the output of Sin(x) is periodic. For each Sin(x) there exists an infinite amount of "x" that could generate that value.

All previous neural networks in the lab are trained to predict Sin(x) from x. For these networks each input is unique. The neural network in this part has a many to one mapping between input and output.

Theory

In neural networks, what do we mean by mini-batch and epoch?

A mini-batch is a small subset of the full data set. It is important that a mini-batch is balanced and representative of the full data set. Mini-batches typically contain 10, 100 or a 1000 data points. An epoch is one complete pass through of the training data. In other words, when a model has processed every training data point once. [p. 125]